DESENVOLVIMENTO MOBILE · MÓDULO 14 · TRILHA FLUTTER · NÍVEL INTERMEDIÁRIO

# Testes e Qualidade

Testes de unidade, de widget, *golden* e de integração, *mocks*, acessibilidade, análise estática, cobertura e integração contínua

**Prof. Dr. Francisco dos Santos Viana**

São Luís – Maranhão – Brasil · 2026

# Objetivos do módulo

## Ao final deste módulo você será capaz de

- Explicar a pirâmide de testes e o que testar em cada nível.
- Testar view models, repositórios e *streams* com *fakes* e `mocktail`.
- Escrever testes de widget, de acessibilidade e *golden*.
- Automatizar fluxos completos com `integration_test` e Patrol.
- Configurar análise estática, formatação e cobertura.
- Rodar tudo a cada *push* com GitHub Actions.

# Roteiro

## Neste módulo

- Fundamentos de teste de software
- Testes no Flutter: visão geral
- Testes de unidade
- Testes de widget
- Testes *golden*
- Testes de integração
- Qualidade contínua
- Fechamento

# Fundamentos de teste de software

# Erro, defeito e falha

## Três palavras que não são sinônimos

**Erro** (*error*): engano cometido por uma pessoa. **Defeito** (*defect*, *bug*): imperfeição no código ou em outro artefato, consequência do erro. **Falha** (*failure*): comportamento incorreto observado quando o defeito é executado.




## Consequência

Um defeito pode ficar anos sem causar falha, se a linha nunca for executada nas condições certas. Testar é **provocar** essas condições.

### Um pouco de história

Em 1947, a equipe do computador Harvard Mark II, da qual Grace Hopper fazia parte, achou uma mariposa presa num relé e a colou no diário de bordo. A palavra *bug* já existia, mas o episódio a popularizou.

# Para que serve testar

## Objetivos dos testes

- Encontrar defeitos antes do usuário.
- Dar confiança para entregar.
- Impedir que defeitos corrigidos voltem (**regressão**).
- Documentar, em código executável, como o sistema deve se comportar.
- Permitir **refatorar** sem medo.

### Princípio fundamental

Testes mostram que defeitos **existem**; nunca provam que **não existem**. A ideia é de Edsger Dijkstra.

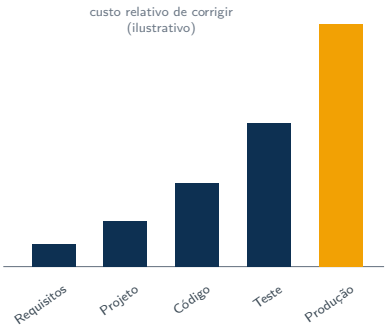
## Teste exaustivo é impossível

Uma função com dois inteiros de 32 bits tem $2^{64}$ combinações de entrada, cerca de $1{,}8 \times 10^{19}$. A um bilhão de testes por segundo, seriam quase 600 anos. Por isso escolhemos casos **representativos** com técnicas de projeto de testes.

### Analogia

O *crash test* de um carro não bate o veículo em todas as velocidades e ângulos: escolhe poucos cenários que representam os acidentes reais.

# O custo de achar tarde




## Por que custa mais

Um defeito achado em produção exige investigar relatos, reproduzir, corrigir, testar de novo, gerar versão, passar pela revisão da loja e esperar os usuários atualizarem. No app, ainda há dados já gravados errado.

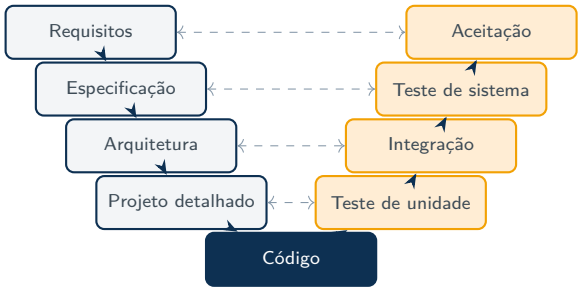
### *Shift left*

Antecipar a qualidade: revisar requisitos, escrever testes junto com o código e rodá-los a cada *commit*.

### Um pouco de história

Barry Boehm mediu esse efeito nos anos 1970 e 1980; os números variam entre projetos, mas a tendência se repete.

# Verificação, validação e níveis de teste




Modelo em V: cada nível de teste confere um nível de especificação.

## Verificação

"Estamos construindo o produto **corretamente**?" Confere o software contra a especificação.

## Validação

"Estamos construindo o produto **certo**?" Confere se o software resolve o problema real do usuário.

### Dica

O teste de aceitação é feito com o cliente: no LabMonitor, os técnicos do laboratório usando o app.

# Caixa-preta: partições e valores-limite

## Teste caixa-preta

Projetado a partir da **especificação**, sem olhar o código: entradas e saídas esperadas.

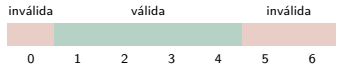
## Partição de equivalência

Divide as entradas em classes que o sistema deve tratar do mesmo jeito; basta um representante de cada classe.

## Análise de valor-limite

Defeitos se concentram nas bordas (< no lugar de <=). Teste o limite e seus vizinhos imediatos.

## Regra: a reserva dura de 1 a 4 horas inteiras




| Casos | Por quê |
|---|---|
| 2 h | representante da classe válida |
| 0 h e 1 h | limite inferior e vizinho |
| 4 h e 5 h | limite superior e vizinho |
| −1 h, “abc” | entradas absurdas |

# Caixa-branca: critérios de cobertura

## Teste caixa-branca

Projetado a partir da **estrutura interna** do código: comandos, decisões e caminhos.

**Pseudocódigo**

```
se temp >= limite E alertaAtivo:
    notificar()
registrar(temp)
```

**Atenção**

Cobertura mede o que foi **executado**, não o que foi **conferido**: um teste sem `expect` também "cobre".

| Critério | Exige, no exemplo |
|---|---|
| **Comandos** | cada linha rodar ao menos uma vez: 1 teste (30 °C, ativo) basta |
| **Decisões** | cada `se` (*branch*) ser verdadeiro e falso: 2 testes |
| **Condições** | cada condição simples verdadeira e falsa: `temp` e `alertaAtivo` variando |
| **Caminhos** | todas as rotas possíveis; cresce exponencialmente com laços e `se`s |

## Hierarquia

Caminhos ⇒ decisões ⇒ comandos: quem satisfaz um critério mais forte satisfaz os mais fracos.

# Dublês de teste

| Dublê | O que é | Exemplo no LabMonitor |
|---|---|---|
| **Dummy** | objeto só para preencher um parâmetro; nunca é usado | um *logger* vazio exigido pelo construtor |
| **Stub** | devolve respostas prontas | repositório que sempre devolve 3 bancadas |
| **Spy** | stub que também registra as chamadas | conta quantas notificações foram enviadas |
| **Mock** | objeto com **expectativas** de interação, verificadas no teste | "`reservar(7)` deve ser chamado uma vez" |
| **Fake** | implementação funcional simplificada | banco em memória, API falsa com dados fixos |

**Dublê de teste**

Objeto que substitui uma dependência real (rede, banco, sensor, relógio) durante o teste.

**Analogia**

Como o dublê de cinema: entra nas cenas perigosas ou caras no lugar do ator.

**Um pouco de história**

A classificação é de Gerard Meszaros, no livro *xUnit Test Patterns* (2007).

# Propriedades de um bom teste

| FIRST | Significado |
|---|---|
| **F**ast | rápido: roda em milissegundos, então roda sempre |
| **I**ndependent | não depende da ordem nem de outros testes |
| **R**epeatable | mesmo resultado em qualquer máquina e horário |
| **S**elf-validating | passa ou falha sozinho, sem alguém olhar a saída |
| **T**imely | escrito junto com o código (ou antes) |

## Estrutura

**AAA**: *Arrange*, *Act*, *Assert*.
**BDD**: *Given*, *When*, *Then* (dado, quando, então).

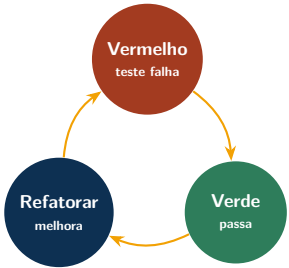
### Teste instável (*flaky*)

Passa e falha sem o código mudar. Causas típicas: depender do relógio real, da rede, da ordem de execução, de esperas fixas (*sleep*) ou de estado compartilhado entre testes.

### Teste também é código

Merece nomes claros, nada de duplicação e revisão. Um bom nome de teste descreve uma **regra**: "reserva sobreposta é recusada".

# TDD, BDD e regressão




## TDD

*Test-Driven Development* (Kent Beck): o teste é escrito **antes** do código e guia o projeto.

**BDD: cenário em linguagem natural (Gherkin)**

```
Funcionalidade: Reserva de bancada
  Cenário: horário já ocupado
    Dado que a bancada B7 está reservada das 14h às 16h
    Quando eu peço a B7 das 15h às 17h
    Então vejo "Horário indisponível"
```

## BDD

*Behavior-Driven Development*: descreve o comportamento em exemplos que clientes, testadores e devs entendem; os cenários viram testes.

## Teste de regressão

Rodar de novo os testes existentes após cada mudança, para garantir que nada que funcionava quebrou. Automatizar é o que torna isso viável.

# Desafios do teste em apps móveis

| Desafio | O que pode dar errado |
|---|---|
| Fragmentação | milhares de modelos Android, versões de sistema, tamanhos e densidades de tela |
| Rede | sem sinal, 3G lento, troca Wi-Fi/móvel no meio de uma requisição |
| Interrupções | ligação, notificação, app enviado ao fundo e encerrado pelo sistema |
| Hardware | câmera, GPS, Bluetooth e sensores diferentes ou ausentes |
| Permissões | usuário nega, concede "só desta vez" ou revoga depois |
| Recursos | bateria fraca, pouca memória, armazenamento cheio |
| Pessoas | idioma, fonte grande, leitor de tela, modo escuro |

## Estratégia

- Automatizar a lógica em testes de unidade e widget (rápidos, sem aparelho).
- Definir uma **matriz de dispositivos** representativa (versões antigas e novas, tela pequena e grande).
- Usar emuladores no dia a dia e aparelhos reais ou *device farms* antes de publicar.
- Simular rede ruim e interrupções.

### Dica

As estatísticas do Android Studio e do console da loja mostram quais versões seus usuários realmente usam.

# Qualidade é mais do que testar

## Modelo de qualidade ISO/IEC 25010

Organiza a qualidade de um produto de software em características como adequação funcional, eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade.

**Dica**

A revisão de 2023 renomeou algumas características (usabilidade virou capacidade de interação) e incluiu *safety*.

## Técnicas que se somam

- **Estáticas** (sem executar): revisão de código, análise estática, *lint*, tipagem forte.
- **Dinâmicas** (executando): testes automatizados e exploratórios.
- **Em produção**: monitoramento de falhas, métricas e *feedback* dos usuários.

Qualidade se **constrói** durante o desenvolvimento; testes confirmam que ela está lá.

# Testes no Flutter: visão geral

# A pirâmide de testes no Flutter




| Tipo | Testa | Roda em |
|---|---|---|
| **Unidade** | uma função, classe, view model ou repositório | VM Dart, ms |
| **Widget** | uma tela ou componente, sem aparelho | ambiente simulado, ms |
| **Golden** | a aparência, pixel a pixel | ambiente simulado |
| **Integração** | o app inteiro em um fluxo real | aparelho ou emulador, s–min |

**Proporção saudável**

Muitos testes de unidade, bastantes de widget e poucos de integração, cobrindo os fluxos críticos (login, reserva).

# Testes de unidade

# Estrutura de um teste

**test/utils/conversao_test.dart**

```dart
import 'package:flutter_test/flutter_test.dart';
import 'package:labmonitor/utils/conversao.dart';

void main() {
  group('celsiusDeBytes', () {
    test('converte centésimos little-endian', () {
      expect(celsiusDeBytes([0xB7, 0x09]), 24.87);
    });

    test('aceita temperatura negativa', () {
      expect(celsiusDeBytes([0x18, 0xFC]), -10.0);
    });

    test('rejeita pacote curto', () {
      expect(() => celsiusDeBytes([0x01]),
          throwsA(isA<FormatException>()));
    });
  });
}
```

```
$ flutter test
00:02 +3: All tests passed!
$ flutter test test/utils/
$ flutter test --name "negativa"
```

## Padrão AAA

**Arrange** (preparar), **Act** (agir), **Assert** (conferir). Um comportamento por teste, com nome que descreve a regra.

### *Matchers* úteis

`equals`, `isTrue`, `hasLength`, `contains`, `closeTo(24.9, 0.1)`, `throwsA`, `isA<T>()`.

# Testando o view model com um *fake*

test/ui/painel_view_model_test.dart

```dart
class RepoFake implements BancadaRepository {
  RepoFake(this.resposta);
  Result<List<Bancada>> resposta;
  @override
  Future<Result<List<Bancada>>> listar() async => resposta;
}
void main() {
  final duas = Ok([bancada('B1'), bancada('B2')]);
  test('carrega ao criar e filtra pelo nome', () async {
    final vm = PainelViewModel(repo: RepoFake(duas));
    await Future<void>.delayed(Duration.zero);   // fim do load
    vm.filtrar('B2');
    expect(vm.visiveis.single.nome, 'B2');
  });
  test('expõe a falha do repositório', () async {
    final repo = RepoFake(Falha(Exception('offline')));
    final vm = PainelViewModel(repo: repo);
    await Future<void>.delayed(Duration.zero);
    expect(vm.carregar.falhou, isTrue);
  });
}
```

**_Fake_**

Implementação simples e funcional da interface, escrita por você. Controla exatamente o que o repositório devolve.

**Dica**

A arquitetura do Módulo 08 se paga aqui: o view model recebe a interface, e o teste dispensa rede, banco e Firebase.

**Atenção**

Teste comportamento observável (estado exposto), não detalhes internos.

# *Mocks* com `mocktail`

**Verificando interações**

```dart
import 'package:mocktail/mocktail.dart';

class RepoMock extends Mock implements BancadaRepository {}

void main() {
  test('tentar de novo chama o repositório outra vez',
      () async {
    final repo = RepoMock();
    when(() => repo.listar()).thenAnswer(
        (_) async => Falha(Exception('offline')));

    final vm = PainelViewModel(repo: repo);
    await Future<void>.delayed(Duration.zero);
    await vm.carregar.execute();

    verify(() => repo.listar()).called(2);
  });
}
```

### *Fake* ou *mock*?

**Fake**: quando importa o *resultado*.
**Mock**: quando importa a *interação* (foi chamado? quantas vezes? com quais argumentos?).

### Dica

`mocktail` não precisa de geração de código. Para argumentos com `any()` de tipos próprios, chame `registerFallbackValue` no `setUpAll`.

# *Streams* e tempo

**Streams**

```
test('converte os pacotes do BLE', () {
  final pacotes = Stream.fromIterable([
    [0xB7, 0x09],
    [0x00, 0x0A],
  ]);
  expect(pacotes.map(celsiusDeBytes),
      emitsInOrder([24.87, 25.6, emitsDone]));
});
```

**Stream matchers**

`emits`, `emitsInOrder`, `emitsThrough`, `emitsError` e `emitsDone` descrevem a sequência esperada.

**Tempo controlado (fake_async)**

```
test('busca espera parar de digitar', () {
  fakeAsync((tempo) {
    final repo = RepoContador();
    final vm = BuscaViewModel(repo: repo);
    vm.digitar('B');
    vm.digitar('B7');
    tempo.elapse(
        const Duration(milliseconds: 299));
    expect(repo.buscas, 0);
    tempo.elapse(
        const Duration(milliseconds: 1));
    expect(repo.buscas, 1);
  });
});
```

**Dica**

Nada de `sleep`: o tempo avança na hora, e o teste fica rápido e determinístico.

# Testes de widget

# Testando uma tela

**test/ui/painel_screen_test.dart**

```dart
testWidgets('mostra o erro e tenta de novo', (tester) async {
  final repo = RepoFake(Falha(Exception('offline')));
  final vm = PainelViewModel(repo: repo);

  await tester.pumpWidget(
      MaterialApp(home: PainelScreen(vm: vm)));
  await tester.pump();                    // conclui o load
  expect(find.text('Tentar de novo'), findsOneWidget);

  repo.resposta = Ok([bancada('B1')]);
  await tester.tap(find.text('Tentar de novo'));
  await tester.pump();

  expect(find.text('B1'), findsOneWidget);
  expect(find.text('Tentar de novo'), findsNothing);
});
```

**`WidgetTester`**

Monta a árvore de widgets sem aparelho. `pump` desenha um quadro; `pumpAndSettle` desenha até as animações terminarem.

**Atenção**

O tempo é simulado: `Future`s e `Timer`s só avançam com `pump`. Esquecer o `pump` é a causa mais comum de teste falhando.

# Encontrar, agir e conferir

| Uso | API |
|---|---|
| Encontrar | `find.text`, `find.byType`, `find.byKey`, `find.byIcon`, `find.bySemanticsLabel` |
| Agir | `tap`, `longPress`, `enterText`, `drag`, `scrollUntilVisible` |
| Conferir | `findsOneWidget`, `findsNothing`, `findsNWidgets(3)`, `findsAtLeast(1)` |

**Dica**

Dê `Key`s estáveis aos campos importantes (`Key('email')`): o teste não quebra quando o texto muda.

**Acessibilidade automática**

```
testWidgets('painel acessível', (tester) async {
  final h = tester.ensureSemantics();
  await tester.pumpWidget(appDeTeste());
  await expectLater(tester,
      meetsGuideline(androidTapTargetGuideline));
  await expectLater(tester,
      meetsGuideline(labeledTapTargetGuideline));
  await expectLater(tester,
      meetsGuideline(textContrastGuideline));
  h.dispose();
});
```

**O que é verificado**

Alvos de toque com 48 dp, botões com rótulo para leitores de tela e contraste de texto.

# Testes *golden*

# Comparando imagens

test/ui/cartao_bancada_golden_test.dart

```dart
testWidgets('cartão de bancada ocupada', (tester) async {
  await tester.pumpWidget(MaterialApp(
    theme: temaLab,
    home: Center(child: CartaoBancada(
        bancada: bancada('B7', livre: false))),
  ));
  await expectLater(find.byType(CartaoBancada),
      matchesGoldenFile('goldens/cartao_ocupada.png'));
});
```

```
$ flutter test --update-goldens   # gera/atualiza
$ flutter test                    # compara
```

### *Golden test*

Renderiza o widget, salva uma imagem de referência e, nas execuções seguintes, falha se algum pixel mudar. Pega regressões visuais que nenhum `expect` pegaria.

### Atenção

Fontes e antialiasing variam entre sistemas: gere e compare os *goldens* no mesmo ambiente (de preferência no CI).

### Dica

Teste os estados: claro e escuro, texto grande, carregando, erro.

# Testes de integração

# O app inteiro no aparelho

integration_test/reserva_test.dart

```dart
import 'package:integration_test/integration_test.dart';
import 'package:labmonitor/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('aluno reserva uma bancada', (tester) async {
    app.main();                  // aponta para os emuladores
    await tester.pumpAndSettle();
    await tester.enterText(find.byKey(const Key('email')),
        'aluno@teste.dev');
    await tester.enterText(find.byKey(const Key('senha')),
        'segredo123');
    await tester.tap(find.text('Entrar'));
    await tester.pumpAndSettle();
    await tester.tap(find.text('B7'));
    await tester.tap(find.text('Reservar'));
    await tester.pumpAndSettle();
    expect(find.text('Reserva confirmada'), findsOneWidget);
  });
}
```

```
$ flutter test integration_test
```

**Mesmo jeito de escrever**

A API é a do teste de widget, mas roda no aparelho real ou emulador, com plugins, rede e banco de verdade.

**Dados de teste**

Nunca use o backend de produção. Use o Emulator Suite (Módulo 12) ou uma API de homologação.

# Além do Flutter: Patrol e nuvem de aparelhos

**Patrol: interagindo com o sistema**

```
patrolTest('lê QR code após permitir a câmera',
    ($) async {
  await $.pumpWidgetAndSettle(const LabMonitorApp());
  await $('Ler QR code').tap();
  // diálogo nativo do Android/iOS
  if (await $.platform.mobile
      .isPermissionDialogVisible()) {
    await $.platform.mobile.grantPermissionWhenInUse();
  }
  await $(MobileScanner).waitUntilVisible();
});
```

**Limite do `integration_test`**

Ele só enxerga widgets Flutter. Diálogos de permissão, notificações e configurações do sistema ficam de fora.

**Patrol**

Estende os testes com interação nativa e uma sintaxe mais curta de *finders*.

**Firebase Test Lab**

Roda os testes de integração em dezenas de modelos reais de Android e iOS na nuvem.

# Qualidade contínua

# Análise estática, formatação e cobertura

**analysis_options.yaml**

```yaml
include: package:flutter_lints/flutter.yaml

analyzer:
  language:
    strict-casts: true
    strict-raw-types: true
  errors:
    unawaited_futures: error

linter:
  rules:
    - prefer_final_locals
    - avoid_print
    - unawaited_futures
```

```
$ dart format .
$ flutter analyze
No issues found!
$ flutter test --coverage
$ genhtml coverage/lcov.info -o coverage/html
```

## Três guardas

**Formatador**: estilo único, sem discussão.
**Analisador**: erros e más práticas antes de rodar.
**Cobertura**: quais linhas os testes executam.

**Atenção**

Cobertura alta não garante bons testes. Use-a para achar código **sem** teste, não como meta.

# Integração contínua com GitHub Actions

**.github/workflows/qualidade.yml**

```yaml
name: qualidade
on: [push, pull_request]

jobs:
  testes:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true
      - run: flutter pub get
      - run: dart format --set-exit-if-changed .
      - run: flutter analyze
      - run: flutter test --coverage
```

## Integração contínua

A cada *push* e *pull request*, um servidor limpo baixa o código, formata, analisa e testa. Se algo falha, o PR fica vermelho e não deve ser integrado.

**Dica**

Proteja o *branch* `main`: exija o *check* verde para aceitar o PR (Módulo 03).

**Dica**

O Módulo 18 estende o pipeline com *build* e publicação automática.

# Estratégia: o que testar primeiro

## Onde investir primeiro

Comece pelo que mais dói quando quebra e é mais barato de testar: regras de negócio e conversões de dados. Telas e fluxos completos vêm depois, cobrindo os caminhos críticos.

### Bug corrigido = teste novo

Antes de corrigir um bug, escreva o teste que o reproduz. Ele nunca mais volta.

| Prioridade | Exemplos no LabMonitor |
|---|---|
| Regras de negócio | conflito de reserva, limite de alerta |
| Conversões | bytes do BLE, JSON da API |
| View models | carregando, erro, vazio, filtro |
| Telas críticas | login, reserva, alertas |
| Fluxos ponta a ponta | entrar → reservar → cancelar |

### Atenção

Não teste o framework (se o `Text` mostra texto) nem pacotes de terceiros.

# Fechamento

# Resumo

## Resumo

- Pirâmide: muitos testes de unidade, bastantes de widget, poucos de integração.
- *Fakes* para controlar resultados; `mocktail` para verificar interações.
- `WidgetTester` monta telas sem aparelho; *goldens* protegem a aparência.
- `integration_test` e Patrol exercitam o app real; Test Lab multiplica aparelhos.
- Formatador, analisador, cobertura e CI mantêm a qualidade a cada *push*.

# Glossário (1/2)

| Termo | Significado |
|---|---|
| **Teste de unidade** | verifica uma unidade isolada (função, classe, view model), sem dependências reais |
| **Teste de widget** | monta um widget ou tela em ambiente simulado e interage com ele |
| **Teste *golden*** | compara a renderização com uma imagem de referência salva |
| **Teste de integração** | executa o app completo em aparelho ou emulador |
| **Teste de regressão** | reexecução dos testes após mudanças para garantir que nada quebrou |
| **Dublê de teste** | substituto de uma dependência real: *dummy*, *stub*, *spy*, *mock* ou *fake* |
| ***Fake*** | implementação simplificada e funcional (repositório em memória) |
| ***Mock*** | dublê que verifica as interações esperadas (chamadas e argumentos) |

# Glossário (2/2)

| Termo | Significado |
|---|---|
| ***Matcher*** | objeto que descreve o valor esperado em um `expect` (`equals`, `throwsA`) |
| ***Finder*** | localiza widgets na árvore durante o teste (`find.text`, `find.byKey`) |
| `WidgetTester` | controla o teste de widget: monta, desenha quadros e simula toques |
| `pump` / `pumpAndSettle` | avança um quadro / avança até não haver animações pendentes |
| **Cobertura** | proporção do código executada pelos testes (comandos, decisões, condições) |
| **Análise estática** | inspeção do código sem executá-lo (`flutter analyze`, *lints*) |
| **Teste instável (*flaky*)** | teste que passa ou falha sem mudança no código |
| **CI** | integração contínua: compilar e testar automaticamente a cada *push* |

# Questões de revisão

1. Diferencie erro, defeito e falha usando um exemplo do LabMonitor.
2. Por que testes não provam a ausência de defeitos? Que técnicas reduzem o número de casos sem perder poder de detecção?
3. A regra é "alerta quando a temperatura for maior ou igual a 30 °C". Quais valores você testaria e por quê?
4. Mostre um exemplo em que 100% de cobertura de comandos não implica 100% de cobertura de decisões.
5. Quando usar um *fake* e quando usar um *mock*?
6. O que um teste de widget não consegue verificar e que exige um teste de integração?
7. Que cuidado os testes *golden* exigem no CI?
8. Cite três causas de testes instáveis e como evitá-las.

# Exercícios

## Exercício 1 — Unidade

Escreva testes para a regra "uma bancada não pode ter duas reservas sobrepostas", incluindo os limites de horário.

## Exercício 2 — Widget e golden

Teste os estados carregando, vazio, erro e lista do painel e crie *goldens* nos temas claro e escuro.

## Exercício 3 — CI

Crie o *workflow* do GitHub Actions e torne-o obrigatório para *pull requests*.

**LabMonitor — Etapa 14**

Testes dos view models e conversões, testes de widget das telas principais, um *golden* por componente e um teste de integração do fluxo de reserva no CI.

# Para aprofundar

- Testes no Flutter: `https://docs.flutter.dev/testing/overview`
- Testes de integração: `https://docs.flutter.dev/testing/integration-tests`
- mocktail: `https://pub.dev/packages/mocktail`
- Patrol: `https://patrol.leancode.co`
- Regras do analisador: `https://dart.dev/tools/linter-rules`
- Firebase Test Lab: `https://firebase.google.com/docs/test-lab`

# Respostas das questões de revisão

1. **Erro**: o dev escreve > em vez de >=. **Defeito**: a regra de alerta fica errada no código. **Falha**: a exatamente 30 °C o alerta não aparece.
2. O espaço de entradas é grande demais para testar tudo; um teste só mostra o comportamento nos casos executados. Partição de equivalência e valores-limite escolhem casos representativos.
3. 29,9 (logo abaixo, sem alerta), 30,0 (no limite, com alerta) e 30,1 (acima), além de um valor típico de cada classe e entradas inválidas (sem leitura).
4. `se cond: x()` seguido de `y()`: um teste com `cond` verdadeira executa todas as linhas, mas o lado falso da decisão nunca é exercitado.
5. *Fake* quando importa o resultado (dados devolvidos); *mock* quando importa a interação (se e como uma dependência foi chamada).
6. Plugins nativos, permissões, rede e banco reais, desempenho no aparelho e o fluxo entre telas com o app inteiro.
7. Gerar e comparar as imagens no mesmo ambiente (sistema, fontes, versão do Flutter), de preferência no próprio CI, para evitar diferenças de renderização.
8. Relógio real, rede real e esperas fixas; ordem de execução e estado compartilhado. Evite com *fakes*, tempo simulado (`fakeAsync`, `pump`) e `setUp` que recria o estado a cada teste.

# Fim do Módulo 14

**Próximo:** Módulo 15 — Publicação: Google Play e App Store.